
Lab0 GitLab 实验报告

姓名：陈星沅

学号：25800190027

一、文档要求回答的问题

1. 你之前有过多人的协同开发经历吗？如果有，你们是使用什么方式分工协作的？ 无。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

1. **精细控制**：允许只把部分修改（而非全部改动）放入暂存区，避免误提交。
2. **保证逻辑完整顺畅**：将相关修改积攒后一次性提交，保证每一次 Commit 都是一个完整的逻辑变更。
3. **提供反悔机制**：在正式提交前，可以撤出误添加的文件，避免污染提交历史。

2. `git branch` 和 `git branch -a` 的区别是什么？

- `git branch`：仅查看本地分支。
- `git branch -a`：查看所有分支（包含本地分支和以 `remotes/origin/` 开头的远程分支）。

4. 阅读“Commit Message 规范”和“Git Flow 分支控制”网页的内容概括与思考

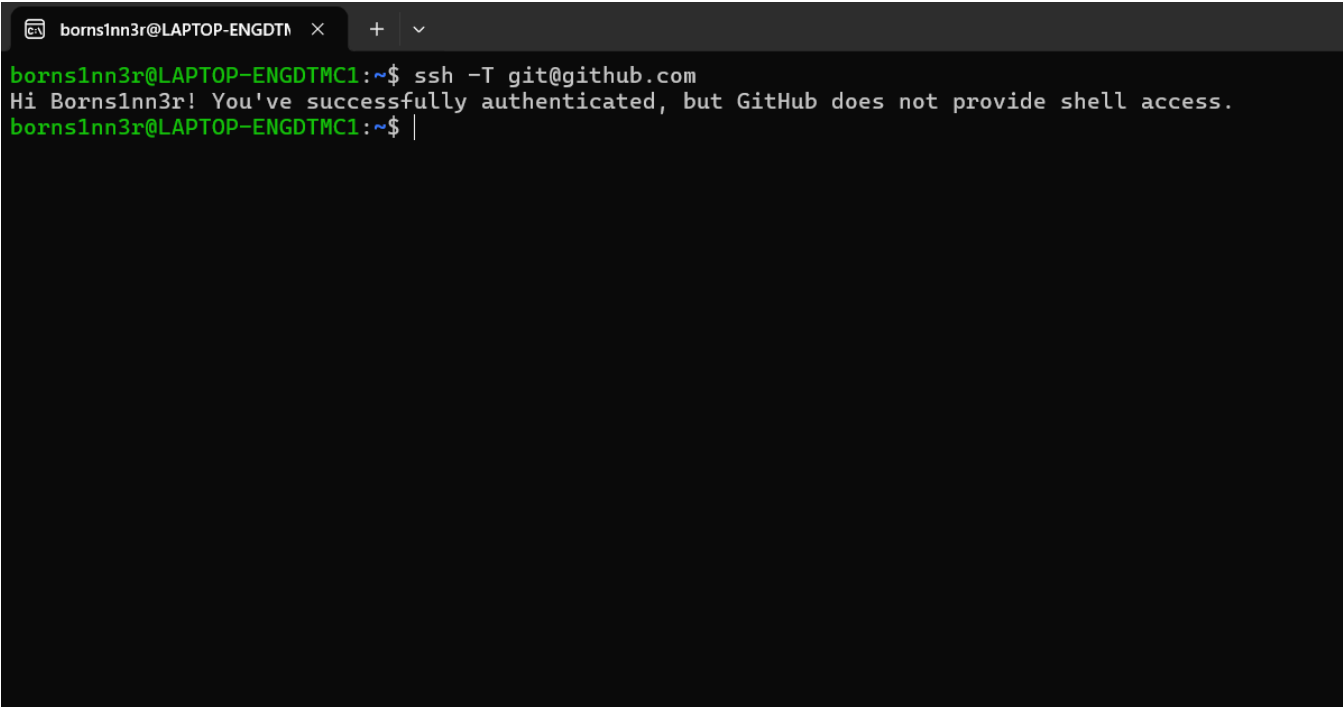
- **网页概括**：Commit Message 规范要求提交信息包含类型（如 `feat`、`fix`）和简短描述，以保证历史清晰可读；Git Flow 分支控制则通过 `main`、`develop`、`feature`、`release`、`hotfix` 等分支的合并规则，实现不同开发阶段的有效隔离。
 - **为什么要学习 Git**：Git 是现代软件工程协作的基石。对个人而言，它是随时可回退的“后悔药”；对团队而言，它让多人并行开发成为可能，减少了代码冲突和沟通成本；对职业发展而言，熟练使用 Git 是程序员必备的基本功。
-

二、实验步骤

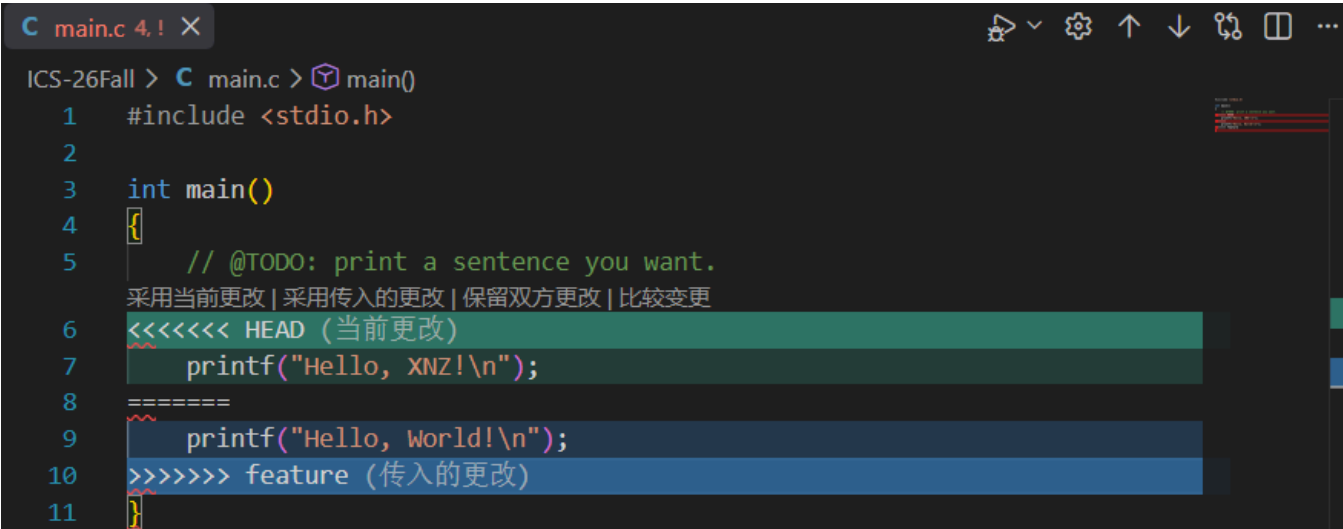
1. **环境配置与连接**：在 WSL 中配置 Git，生成 SSH 密钥并添加到 GitHub。
 2. **克隆仓库**：将远程仓库克隆至本地 WSL 环境中。
 3. **创建并切换分支**：在 VS Code 左下角点击 `main`，创建并切换到新分支 `feature`。
 4. **在 feature 分支修改并提交**：在 `main.c` 文件中修改特定行，使用 VS Code 源代码管理面板暂存并提交。
 5. **切回 main 分支并修改同一行**：切回 `main` 分支后，修改 `main.c` 的同一行代码（这是制造冲突的关键），并同样进行暂存和提交。
 6. **合并分支并解决冲突**：在 `main` 分支下将 `feature` 分支合并过来，触发冲突。在 VS Code 的冲突可视化界面中，选择保留两处修改或手动编辑，去除冲突标记（`<<<<<<<<`、`=====`、`>>>>>>>>`）。
 7. **提交解决结果**：将解决冲突后的代码暂存，提交合并结果。
-

三、必要的截图

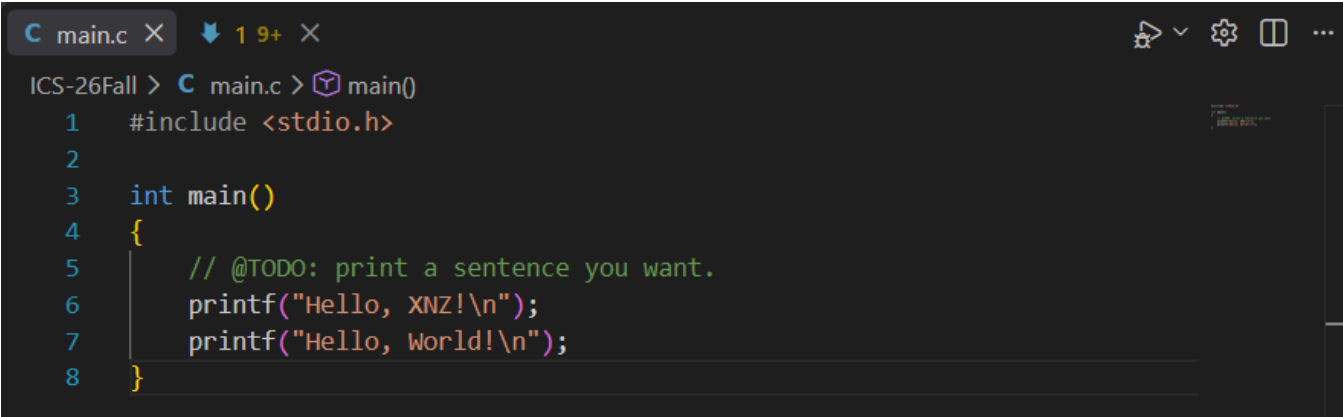
截图 1：在 WSL 中通过 SSH 成功连接 GitHub 的终端截图

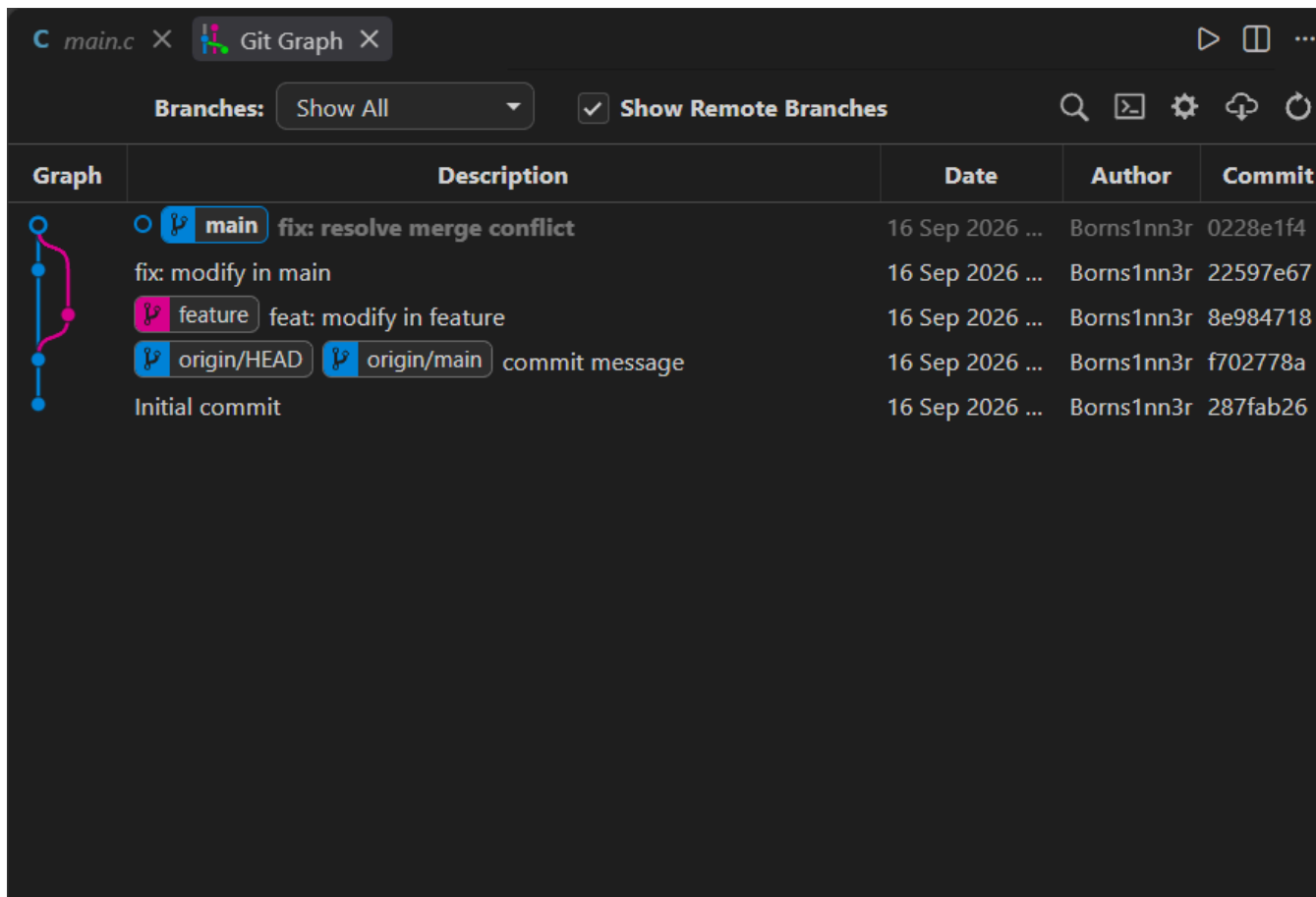


截图 2：VS Code 中的代码冲突界面截图



截图 3：解决冲突后的最终提交截图





四、建议

- **关于环境配置**：WSL 下的网络环境有时较为复杂（如 IPv6 超时、TLS 连接中断等），建议在遇到网络问题时优先尝试配置 SSH 或修改镜像源。
- **关于工具使用**：VS Code 提供的 Git 图形化界面大大降低了初学者解决冲突的门槛，建议多尝试结合 GUI 和命令行学习 Git 原理。